

Uma Abordagem Baseada em Atenção para o Problema de Coletas e Entregas Dinâmicas

Antonio Neto, Rodrigo Drummond Macedo

Deeptera Sistemas Inteligentes

{antonio, rodrigo}@deeptera.com

Arley Henrique Duque

Framework Digital

arleyduque@frwk.com.br

Sandro Jerônimo de Almeida

Pontifícia Universidade Católica de Minas Gerais

sandro.j@pucminas.br

RESUMO

O Problema de Coletas e Entregas Dinâmicas (*Dynamic Pickup and Delivery Problem - DPDP*) busca criar planos de despacho para uma frota de veículos diante de pedidos revelados dinamicamente, sem conhecimento prévio completo. Além da incerteza temporal, o problema envolve restrições operacionais como capacidade dos veículos, precedência entre coleta e entrega, regras de descarregamento e atrasos acumulados, o que dificulta o uso direto de métodos exatos em cenários de decisão contínua. Este trabalho propõe uma abordagem baseada em Modelo de Atenção treinado via Aprendizado por Reforço e combinado com busca local. O modelo foi adaptado ao DPDP por meio de atributos temporais e operacionais, máscaras de viabilidade e refinamento local das rotas geradas. A avaliação no benchmark ICAPS 2021, frente a heurísticas amplamente utilizadas na literatura de otimização combinatória, indica que a abordagem proposta reduz o custo médio agregado em 98,7% comparado à regra Round-Robin e em 16,3% em relação à Busca Tabu, além de superar o algoritmo genético em 31,2% nos grupos 1–56.

PALAVRAS CHAVE. Problema de Coletas e Entregas Dinâmicas, Modelo de Atenção, Aprendizado por Reforço, Roteamento de Veículos, Inteligência Artificial.

ABSTRACT

The *Dynamic Pickup and Delivery Problem* (DPDP) requires generating dispatch plans for a fleet of vehicles under dynamically revealed requests and incomplete prior information. Beyond temporal uncertainty, the problem includes operational constraints such as vehicle capacity, pickup-and-delivery precedence, unloading order and accumulated delays, which make exact methods difficult to apply in continuous decision settings. This work proposes an Attention Model trained with Reinforcement Learning and combined with local search. The model is adapted to the DPDP through temporal and operational features, feasibility masks and local route refinement. Experiments on the ICAPS 2021 benchmark, against heuristics widely used in the combinatorial optimization literature, indicate that the proposed approach reduces aggregate average cost by 98.7% compared to the Round-Robin rule and by 16.3% relative to Tabu Search, while outperforming the genetic algorithm by 31.2% average reduction in groups 1–56.

KEYWORDS. Dynamic Pickup and Delivery Problem, Attention Model, Reinforcement Learning, Vehicle Routing, Artificial Intelligence.

1. Introdução

O problema de otimização de rotas é um dos temas centrais da pesquisa em otimização combinatória e logística computacional. Em sua forma mais elementar, o *Travelling Salesman Problem* (TSP) [Applegate et al., 2006] consiste em determinar o caminho de menor custo que visita um conjunto de nós exatamente uma vez e retorna à origem — um problema NP-difícil cuja complexidade cresce com o número de cidades. O TSP serve como caso base para uma vasta família de problemas de roteamento com aplicações diretas em manufatura, telecomunicações e distribuição logística.

A incorporação de uma frota de veículos com capacidade finita origina o *Vehicle Routing Problem* (VRP) [Dantzig e Ramser, 1959], no qual o objetivo é minimizar o custo total de rotas que coletivamente atendam a um conjunto de clientes com demandas conhecidas. Extensões canônicas do VRP introduzem restrições de janelas de tempo (*VRPTW*) [Solomon, 1987], frotas heterogêneas (*HFVRP*), múltiplos depósitos (*MDVRP*) e combinações dessas variantes, cada qual modelando aspectos distintos de cenários reais de distribuição urbana e industrial.

Uma subclasse relevante do VRP é o *Pickup and Delivery Problem* (PDP) [Savelsbergh e Sol, 1995], que generaliza o problema ao exigir que cada requisição seja composta por um par de nós — um ponto de coleta (*pickup*) e um ponto de entrega (*delivery*) — com a restrição de precedência de que a coleta deve ocorrer antes da entrega correspondente. O PDP captura cenários de transporte de cargas entre origens e destinos distintos, como operações de redistribuição de contêineres e serviços de compartilhamento de veículos. Variantes como o *PDPTW* [Li e Lim, 2001] acrescentam janelas de tempo estritas a cada nó, e variantes como o *PDPLIFO* [Cordeau et al., 2010] impõem restrições de ordem de carregamento do tipo *Last-In-First-Out* (LIFO) aos veículos.

No escopo do gerenciamento de cadeias de suprimentos de manufatura em larga escala e operações de logística urbana, o *Dynamic Pickup and Delivery Problem* (DPDP) emerge como um desafio de otimização combinatória NP-difícil [Garey e Johnson, 1979; Cai et al., 2023] que é uma variação do Pickup and Delivery Problem (PDP). O DPDP requer o despacho e o roteamento contínuo de uma frota de veículos para atender a requisições de transporte de cargas entre nós de coleta e entrega, cujas informações (distribuição espacial, janelas de tempo, volume de carga) são reveladas de forma estocástica e contínua ao longo do horizonte temporal [Psaraftis, 1988; Hao et al., 2022; Liu et al., 2018].

Instâncias práticas do DPDP são encontradas em diversos setores. Na logística de *e-commerce* e *last-mile delivery*, plataformas de entrega urbana precisam rotear continuamente seus veículos à medida que novos pedidos chegam ao longo do dia [Ulmer et al., 2021]. Em cadeias de suprimentos industriais, fábricas com múltiplos centros de distribuição geram requisições de transporte de paletes entre plantas com janelas de tempo estritas [Su et al., 2022]. Serviços de *ride-sharing* e transporte sob demanda, como táxis compartilhados, enfrentam versões do DPDP onde passageiros solicitam viagens com origens e destinos distintos em tempo real [Bouros et al., 2011]. Plataformas colaborativas de logística, incluindo modelos de *crowdshipping*, também introduzem variantes do DPDP com tempos de viagem dependentes do horário [Stoia et al., 2025]. Em todos esses cenários, a natureza dinâmica das requisições e a necessidade de decisões em tempo real tornam o problema particularmente desafiador.

Historicamente, o DPDP foi abordado sob a ótica de reotimizações periódicas valendo-se de programação matemática inteira mista ou metaheurísticas de busca local [Karami et al., 2020; Ropke e Pisinger, 2006]. Contudo, o aumento da escala do problema torna a convergência de métodos exatos intratável, ao passo que métodos heurísticos sofrem de ineficiência computacional ou miopia na generalização sobre espaços de estados não estacionários.

Este trabalho propõe uma solução utilizando um Modelo de Atenção [Kool et al., 2019].

O objetivo é apresentar a arquitetura de atenção para o problema do DPDP e fazer uma comparação analítica com outros métodos.

O texto está dividido da seguinte forma: A Seção 2 apresenta uma revisão da literatura, traçando a evolução dos algoritmos de DPDP e usos recentes de Modelos de Atenção em problemas de logística e transporte. A Seção 3 detalha a modelagem do Modelo de Atenção adaptado ao DPDP, explicitando os processos de projeção de *embeddings*, extração de contexto global via *Encoder* e decodificação com mascaramento físico contínuo. Na Seção 4, conduzimos comparações dos resultados obtidos com outros algoritmos. Por fim, a Seção 5 sintetiza as conclusões desta pesquisa.

2. Trabalhos Relacionados

Karami et al. [2020] utilizam uma abordagem de reotimizações usando MILP (Mixed Integer Linear Programming) para resolver o problema do DPDP com janelas de tempo, mas não demonstram nenhuma aplicação em ambientes de escala industrial. Devido à natureza NP-difícil do problema, o tempo computacional exigido por métodos exatos torna-os inviáveis para aplicações operacionais. A literatura consolidou o uso de heurísticas e metaheurísticas como a abordagem mais eficiente, e evoluiu consideravelmente a partir delas. A Busca Adaptativa em Grandes Vizinhanças (ALNS) [Ropke e Pisinger, 2006] e suas variantes hibridizadas como IALNS-SA [Ma e Yang, 2025] estabeleceram-se como métodos robustos. Mais recentemente, algoritmos meméticos (método genético com etapas de refinamento locais) [Zhou et al., 2024] e métodos baseados em Busca em Vizinhança Variável (VNS) [Cai et al., 2022] também foram propostos.

A aplicação de técnicas de Aprendizado por Reforço (RL) e redes neurais ao roteamento de veículos tem ganhado relevância crescente. Li et al. [2021] propuseram um *framework* baseado em RL para otimizar instâncias do DPDP em escala industrial, demonstrando que políticas aprendidas podem generalizar para cenários não vistos durante o treinamento. Cordeiro e Pitombeira-Neto [2023] apresentaram uma abordagem baseada em Aprendizado por Reforço Profundo (DRL) para o despacho dinâmico de veículos, utilizando uma formulação orientada a eventos que permite ao agente tomar decisões de roteamento em tempo real. Jiang e Chang [2025] integraram DRL com teoria de filas para otimizar o roteamento dinâmico de última milha. No contexto de entregas com múltiplas transferências, Chen et al. [2021] propuseram o DeepFreight, que combina aprendizado por reforço multi-agente (QMIX) com um otimizador MILP para despacho de frotas. Essas abordagens compartilham a vantagem de substituir regras heurísticas fixas por políticas adaptativas, que aprendem padrões latentes nos dados operacionais.

Paralelamente à evolução das metaheurísticas na pesquisa operacional, os Modelos de Atenção emergiram como uma mudança de paradigma nas arquiteturas de redes neurais profundas [Chaudhari et al., 2021]. Originalmente concebidos para solucionar gargalos no Processamento de Linguagem Natural (NLP), como no trabalho de Bahdanau et al. [2014], os mecanismos de atenção ganharam destaque por sua capacidade de focar seletivamente nos elementos mais relevantes de uma sequência de dados de entrada. Esse avanço permitiu superar limitações de modelos recorrentes tradicionais, especialmente na modelagem de dependências de longo alcance.

Com a introdução da arquitetura Transformer por Vaswani et al. [2017], os mecanismos de atenção passaram a ocupar papel central no processamento sequencial, eliminando completamente a necessidade de recorrência e permitindo maior paralelização e eficiência computacional. A partir desse marco, modelos baseados em atenção, como o BERT [Devlin et al., 2019], consolidaram-se como estado da arte em diversas tarefas de NLP. Posteriormente, a mesma arquitetura Transformer passou a sustentar os *Large Language Models* (LLMs), nos quais o mecanismo de atenção é escalado para grandes volumes de dados e parâmetros.

Além disso, a literatura recente evidencia que os mecanismos de atenção têm sido amplamente aplicados em diferentes domínios além do NLP. Em visão computacional, por exemplo, o

Vision Transformer [Dosovitskiy et al., 2021] e o DETR [Carion et al., 2020] demonstram a eficácia da atenção na modelagem de dependências espaciais e na reformulação de problemas clássicos como classificação e detecção de objetos. De forma mais abrangente, estudos de revisão, como Guo et al. [2022], destacam o uso de mecanismos de atenção em tarefas como segmentação de imagens, reconhecimento de ações, séries temporais, bioinformática e aprendizado multimodal.

O trabalho de Kool et al. [2019] demonstrou que um Modelo de Atenção bidirecional, treinado via Aprendizado por Reforço, pode atuar como um poderoso *Encoder-Decoder* para resolver variações estáticas do VRP. A arquitetura utiliza um *Encoder* Transformer para gerar *embeddings* dos nós do grafo e um *Decoder* autorregressivo que constrói a solução nó a nó, empregando atenção com mascaramento para garantir viabilidade. O treinamento é realizado com o algoritmo REINFORCE e uma *baseline* de *greedy rollout*. Os resultados reportados demonstraram desempenho competitivo com heurísticas especializadas no TSP e no CVRP, com tempos de inferência significativamente menores e capacidade de generalização para instâncias de tamanhos variados.

Este trabalho se insere e expande essa linha de pesquisa ao adaptar a arquitetura de atenção especificamente aos desafios estocásticos do DPDP. Enquanto o modelo original de Kool et al. opera sobre instâncias estáticas com todos os nós conhecidos *a priori*, a adaptação proposta lida com requisições reveladas dinamicamente, incorporando informações temporais (prazos, janelas de tempo) e restrições operacionais (capacidade, LIFO, filas de doca) ao vetor de *features* dos nós. Além disso, o modelo é combinado com uma etapa de busca local VNS para refinar as soluções geradas pela atenção, resultando em uma abordagem híbrida que une a capacidade de generalização das redes neurais com a precisão da otimização local.

3. Metodologia

Esta seção descreve como o problema foi representado e como a política proposta transforma o estado dinâmico do sistema em decisões de despacho. A metodologia combina três componentes principais: a formulação operacional do DPDP, a representação neural de pedidos e veículos por meio de *embeddings*, e a construção de soluções por um Modelo de Atenção refinado por busca local. Com isso, preserva-se a estrutura logística do problema enquanto o modelo aprende relações entre demanda, capacidade, localização e urgência.

3.1. Formulação do problema

O problema é formulado considerando um cenário de despacho contínuo onde existe um conjunto de fábricas F , uma frota de veículos V e um conjunto de pedidos O que surgem dinamicamente ao longo do tempo. Cada pedido $o \in O$ é definido por uma fábrica de coleta p_o , uma fábrica de entrega d_o , uma demanda de carga q_o , um instante de revelação (ou criação) r_o e um prazo limite para entrega l_o .

O problema é dinâmico porque nem todos os pedidos são conhecidos no início. O simulador avança em ciclos de 10 minutos. Em cada ciclo, o algoritmo recebe os pedidos disponíveis, a posição atual dos veículos e as rotas já comprometidas. A decisão do algoritmo é escolher quais pedidos serão atendidos, por quais veículos e em qual ordem.

A solução construída em um ciclo de despacho é denotada por S . A função de custo combina distância total e atraso:

$$C(S) = \sum_{v \in V} D_v(S) + \delta \sum_{o \in O} \max(0, a_o(S) - l_o) \quad (1)$$

onde $D_v(S)$ é a distância percorrida pelo veículo v , $a_o(S)$ é o instante de entrega do pedido o , l_o é seu prazo e $\delta = 10000/3600$. Assim, atrasos recebem uma penalidade alta, enquanto soluções sem atraso são avaliadas principalmente pela distância.

Duas restrições operacionais são consideradas de forma explícita. A primeira é a capacidade: a carga carregada por um veículo não pode exceder sua capacidade. A segunda é a regra LIFO (*Last-In-First-Out*), em que o último item carregado deve ser o primeiro descarregado quando a organização física da carga exige essa ordem [Cordeau et al., 2010].

3.2. Representação da entrada

Antes de aplicar o modelo de atenção, os pedidos são agrupados em blocos. Um bloco é a unidade de decisão usada pelo algoritmo: ele reúne um ou mais itens que compartilham a mesma fábrica de coleta e a mesma fábrica de entrega. Essa escolha reduz o número de decisões individuais e evita que itens naturalmente relacionados sejam tratados como pedidos independentes. Cada bloco é descrito por um vetor de atributos, chamado *embedding* de entrada. Nesse contexto, *embedding* significa apenas uma representação numérica do objeto que será processada pela rede neural.

Cada bloco b é representado por um vetor com informações sobre distância, demanda, prazo e localização:

$$\mathbf{x}_b = [\tilde{d}_p, \tilde{d}_d, \tilde{q}, \tilde{t}_l, \text{lat}_p, \text{lng}_p, \text{lat}_d, \text{lng}_d] \quad (2)$$

onde $\tilde{d}_p$ e $\tilde{d}_d$ são distâncias normalizadas até a coleta e a entrega, $\tilde{q}$ é a demanda normalizada, $\tilde{t}_l$ é o tempo restante até o prazo, e as demais entradas são as coordenadas das fábricas de coleta e entrega.

Cada veículo também é representado por um vetor:

$$\mathbf{x}_v = [\text{lat}_v, \text{lng}_v, \tilde{c}_v, \tilde{t}_v, \tilde{q}_v] \quad (3)$$

onde $\tilde{c}_v$ é a capacidade restante, $\tilde{t}_v$ é o tempo estimado até o veículo ficar disponível e $\tilde{q}_v$ é a carga atual. Todos os valores contínuos são normalizados para manter as escalas comparáveis durante a inferência.

A Figura 1 resume essa relação entre estado dinâmico, representação, política de atenção, treinamento e refinamento local. As setas contínuas indicam o fluxo de inferência no ciclo de despacho, enquanto as setas tracejadas indicam treinamento ou realimentação para o próximo ciclo.

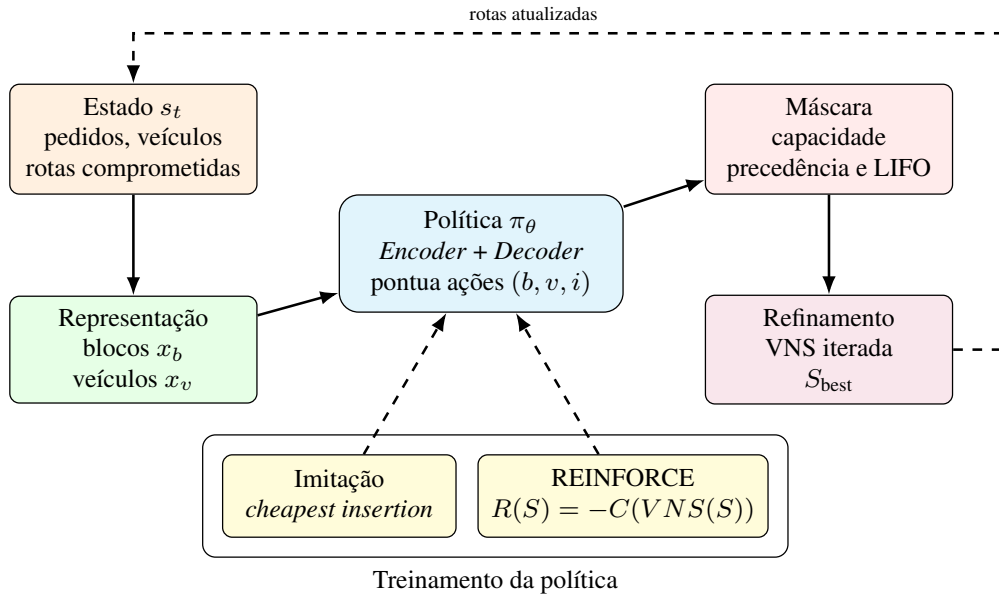


Figura 1: Visão integrada da metodologia.

3.3. Arquitetura do Modelo de Atenção

O modelo segue a estrutura *Encoder-Decoder* usada em Transformers [Vaswani et al., 2017] e em modelos de atenção para roteamento [Kool et al., 2019]. O *Encoder* lê todos os blocos e veículos disponíveis no ciclo de despacho e produz representações contextualizadas, isto é, vetores que levam em conta não apenas o objeto isolado, mas também sua relação com os demais. O *Decoder* usa essas representações para escolher ações de inserção.

Primeiro, os vetores de blocos e veículos são projetados para uma dimensão comum $d_{\text{model}} = 128$:

$$\mathbf{h}_i^{(0)} = W_x \mathbf{x}_i + b_x \quad (4)$$

onde i indexa um bloco ou veículo, $\mathbf{x}_i$ representa o vetor de entrada desse elemento, $\mathbf{h}_i^{(0)}$ é sua representação oculta inicial, e W_x e b_x são os pesos e o viés aprendidos nessa projeção linear. Em seguida, um *Encoder* Transformer com 3 camadas transforma esses vetores em *embeddings* contextualizados.

O mecanismo básico de atenção é:

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V \quad (5)$$

onde Q representa a consulta, isto é, o elemento que busca informação; K representa as chaves, usadas para medir a relevância dos demais elementos; e V representa os valores, que carregam a informação agregada após o cálculo dos pesos de atenção. Uma cabeça de atenção é uma comparação independente entre esses três elementos. No modelo multi-cabeça, esse cálculo é repetido em paralelo por 8 cabeças, permitindo que o modelo observe relações diferentes ao mesmo tempo, como urgência, proximidade e capacidade.

3.4. Treinamento da política

O treinamento é feito em duas etapas. Primeiro, o modelo aprende por imitação a reproduzir decisões de uma heurística de inserção mais barata (*cheapest insertion*) [Solomon, 1987], que escolhe a posição que causa o menor aumento imediato de custo. Essa etapa fornece uma política inicial estável e evita que o treinamento comece a partir de decisões aleatórias. Seja π_θ a política do modelo, parametrizada por θ , que atribui probabilidade às ações possíveis. Em cada passo de decisão t , o estado observado é s_t e a ação escolhida pela heurística é a_t^* . A perda de imitação é:

$$\mathcal{L}_{IL}(\theta) = - \sum_t \log \pi_\theta(a_t^* | s_t) \quad (6)$$

onde $\mathcal{L}_{IL}$ é a perda de aprendizado por imitação. Minimizar essa perda aumenta a probabilidade das ações escolhidas pela heurística nos estados correspondentes.

Para evitar que a política seja ajustada diretamente às instâncias usadas na avaliação final, o treinamento utiliza instâncias sintéticas geradas no mesmo formato do benchmark, com variações no número de pedidos, veículos, fábricas e padrão temporal de chegada dos pedidos. Foram criadas 56 instâncias para treino e 8 instâncias para teste.

Depois, a política é refinada com REINFORCE [Williams, 1992], um método clássico de gradiente de política em Aprendizado por Reforço [Sutton e Barto, 2018]. Durante esse refinamento, a solução gerada pela política passa pela VNS antes da avaliação. Assim, a recompensa usada é o negativo do custo da solução após o refinamento local:

$$R(S) = -C(\text{VNS}(S)) \quad (7)$$

onde $VNS(S)$ representa a solução obtida após aplicar a busca local sobre a solução inicial S construída pela política. A atualização da política segue:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[(R(S) - b) \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \quad (8)$$

onde $J(\theta)$ é o objetivo esperado da política, a_t é a ação amostrada pelo modelo no passo t , $\mathbb{E}[\cdot]$ indica a média esperada sobre as soluções geradas e b é uma *baseline* usada para reduzir a variância do gradiente. Na prática, isso faz com que ações que produzem soluções melhores que a referência se tornem mais prováveis, enquanto ações associadas a soluções piores perdem probabilidade. Como a recompensa considera o custo pós-VNS, o treinamento favorece decisões iniciais que não apenas têm baixo custo imediato, mas também conduzem a regiões do espaço de soluções que a busca local consegue refinar com maior eficiência.

A configuração principal do Modelo de Atenção utilizada nos experimentos é resumida na Tabela 1.

Tabela 1: Configuração principal do Modelo de Atenção.

Parâmetro	Valor
Dimensão dos <i>embeddings</i>	128
Camadas do <i>Encoder</i>	3
Cabeças de atenção	8
Épocas de imitação	200
Épocas de REINFORCE	50

Os valores de dimensão dos *embeddings*, número de camadas do *Encoder* e número de cabeças de atenção seguem a configuração original proposta por Kool et al. [2019] para problemas de roteamento, enquanto o número de épocas de imitação e de REINFORCE foi definido empiricamente por meio de avaliação no conjunto de instâncias sintéticas de treino.

3.5. Decodificação e construção da solução

Durante a inferência, o *Decoder* avalia ações do tipo:

$$\alpha = (b, v, i) \quad (9)$$

onde b é o bloco a inserir, v é o veículo escolhido e i é a posição de inserção na rota. Para cada ação viável, o modelo calcula uma pontuação u_{α} . A ação escolhida é:

$$\alpha^* = \arg \max_{\alpha \in A_{\text{viável}}} u_{\alpha} \quad (10)$$

A máscara garante que $A_{\text{viável}}$ contenha apenas inserções que respeitam capacidade, ordem de coleta antes da entrega e restrições LIFO. Na construção inicial, usamos uma pequena busca em feixe (*beam search*) [Lowerre, 1976], mantendo alternativas parciais promissoras para reduzir decisões míopes sem enumerar todas as rotas.

3.6. Busca local e VNS iterada

Depois que o modelo de atenção constrói uma solução inicial, aplicamos uma busca local VNS (*Variable Neighborhood Search*) [Mladenović e Hansen, 1997]. A escolha pela VNS, em vez de uma Busca em Grandes Vizinhanças (LNS) ou sua variante adaptativa (ALNS) [Ropke e Pisinger, 2006], deve-se à estrutura das vizinhanças exploradas: as operações consideradas ($\mathcal{N}_1 - \mathcal{N}_4$) atuam

sobre blocos e rotas já definidos pela política de atenção, favorecendo movimentos estruturados e de baixo custo computacional por iteração, em vez de destruição e reconstrução de grandes porções da solução. A ideia da VNS é explorar diferentes tipos de vizinhança da solução atual: em vez de depender de um único operador de melhoria, a busca alterna entre mudanças simples e mudanças um pouco mais estruturadas, o que ajuda a escapar de soluções localmente boas, mas ainda melhoráveis. Esse tipo de busca já foi aplicado diretamente a variantes práticas do DPDP [Cai et al., 2022].

Seja $\mathcal{N}_k(S)$ a vizinhança k da solução atual. A busca avalia soluções candidatas $S' \in \mathcal{N}_k(S)$ e aceita apenas movimentos que reduzem a função objetivo:

$$S \leftarrow S' \quad \text{se} \quad C(S') < C(S) \quad (11)$$

As vizinhanças usadas são:

- $\mathcal{N}_1$ - Realocação de bloco: remove um bloco de sua posição atual e testa sua inserção em outras posições e veículos.
- $\mathcal{N}_2$ - Troca entre veículos: troca dois blocos pertencentes a veículos distintos, respeitando capacidade e LIFO.
- $\mathcal{N}_3$ - 2-opt: inverte um trecho da sequência de blocos de um veículo para reduzir distância, seguindo a lógica clássica de melhoria local em roteamento [Lin, 1965].
- $\mathcal{N}_4$ - Agrupamento por rota: move grupos consecutivos com mesma coleta e entrega, preservando rotas com alta concentração de pedidos semelhantes.

A Figura 2 ilustra a VNS. Cada vizinhança tenta melhorar a solução atual; quando há melhoria, a busca reinicia a partir da solução atualizada. Quando não há melhoria, o índice k avança para a próxima vizinhança. Se todas as vizinhanças forem testadas, uma perturbação controlada inicia uma nova rodada enquanto houver tempo restante.

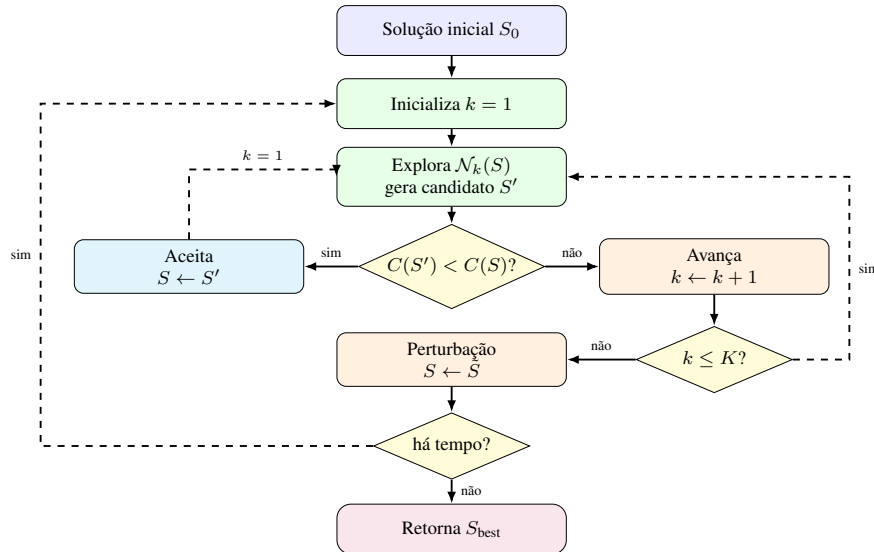


Figura 2: Fluxo iterativo da VNS.

Também usamos uma versão iterada da VNS. Quando nenhuma vizinhança melhora a solução, uma perturbação controlada é aplicada para gerar uma solução próxima $\tilde{S}$. Em seguida, a

VNS é executada novamente a partir de $\tilde{S}$. Apenas a melhor solução encontrada é mantida:

$$S_{\text{best}} = \arg \min_{S \in \mathcal{P}} C(S) \quad (12)$$

onde $\mathcal{P}$ é o conjunto de soluções visitadas durante as rodadas de refinamento. Essa etapa usa o tempo computacional restante para explorar alternativas próximas, sem transformar o método em força bruta, pois apenas um subconjunto pequeno de movimentos viáveis é testado.

4. Resultados

A avaliação experimental utiliza o benchmark ICAPS 2021 [Hao et al., 2022], composto por 64 instâncias que representam diferentes cenários de operação logística. Essas instâncias são organizadas em 8 grupos de 8 instâncias cada, classificados por ordem crescente de complexidade. O que diferencia um grupo do outro é a escala do problema: instâncias nos primeiros grupos possuem poucos veículos e pedidos (cenário de baixa densidade), enquanto os grupos finais (como 57–64) apresentam centenas de veículos e milhares de pedidos revelados ao longo do tempo, exigindo maior eficiência computacional e capacidade de generalização do modelo. Cada instância é dividida em 144 ciclos de 10 minutos (totalizando um dia operacional). Em cada ciclo, novas demandas podem aparecer, e o algoritmo deve produzir uma nova decisão antes do próximo instante de atualização.

Os resultados comparam o Modelo de Atenção proposto com a Busca Tabu, o algoritmo genético e a regra Round-Robin. Para avaliar o desempenho dos algoritmos, considerou-se duas funções objetivo principais que refletem os custos logísticos e operacionais: f_1 , que representa a distância total percorrida pela frota, e f_2 , que quantifica o atraso total acumulado no atendimento dos pedidos.

O custo total de uma solução para uma instância i e o custo médio por grupo são definidos por:

$$C_i = f_{1,i} + \lambda f_{2,i} \quad (13) \quad \bar{C}_G = \frac{1}{|G|} \sum_{i \in G} C_i \quad (14)$$

em que $\lambda = 10000/3600$ é o fator de penalização aplicado ao atraso. Dessa forma, o objetivo central dos algoritmos é minimizar $\bar{C}_G$. Os experimentos foram executados em um processador Intel Core i7-13650HX com 15,69 GB de RAM.

Tabela 2: Comparação entre modelos.

Grupo	Atenção		Busca Tabu		Genético		Round-Robin	
	$\bar{C}_G$	Tempo(s)	$\bar{C}_G$	Tempo(s)	$\bar{C}_G$	Tempo(s)	$\bar{C}_G$	Tempo(s)
1–8	1.0279e+03	144.27	1.0102e+03	195.84	1.1902e+03	168.87	8.2200e+04	83.88
9–16	8.4412e+04	296.29	1.9851e+05	251.66	2.9962e+05	202.10	3.0523e+06	119.31
17–24	9.4222e+02	372.85	1.0625e+03	268.70	1.0677e+03	205.78	1.8709e+06	105.06
25–32	1.6910e+04	1386.39	5.1047e+04	316.63	8.9995e+04	209.60	3.4348e+07	158.79
33–40	1.3019e+04	742.28	1.3641e+04	324.10	1.1541e+04	225.98	4.3941e+07	149.79
41–48	2.6890e+05	1008.74	3.3283e+05	452.14	6.4500e+05	232.93	4.3177e+08	305.77
49–56	4.7752e+06	1581.47	6.1441e+06	521.10	6.4574e+06	269.91	9.2273e+08	428.01
57–64	3.5008e+07	1319.68	4.1273e+07	834.40	3.1055e+07	291.36	1.8643e+09	767.46
Média 1–56	7.3720e+05	790.33	9.6317e+05	332.88	1.0723e+06	216.45	2.0540e+08	192.94
Média 1–64	5.0210e+06	856.50	6.0019e+06	395.57	4.8201e+06	225.82	4.1276e+08	264.76

A regra Round-Robin é utilizada como uma política sequencial sem otimização explícita, incapaz de antecipar efeitos futuros de capacidade, prazos e posicionamento da frota. Como esperado, a abordagem proposta supera essa regra em todos os grupos avaliados, indicando que a

incorporação de representação neural, máscaras de viabilidade e refinamento local produz ganhos em relação a uma política puramente reativa.

Em relação ao algoritmo genético, o Modelo de Atenção obtém custo médio 31,2% menor nos grupos 1–56, vencendo em seis dos sete grupos considerados. Esse resultado sugere que a política aprendida gera boas soluções iniciais para a busca local na maioria das escalas. A principal limitação ocorre no grupo 57–64, em que o algoritmo genético apresenta melhor desempenho. Essa queda pode ser atribuída ao aumento do número de veículos, pedidos e ações viáveis, que amplia o espaço de decisão do *Decoder*; à menor capacidade de generalização da política em cenários de escala extrema, devido ao número limitado de instâncias sintéticas utilizadas no treinamento; e às limitações do espaço de vizinhança explorado pela VNS.

A comparação com a Busca Tabu reforça o mesmo padrão. O Modelo de Atenção apresenta menor custo em sete dos oito grupos, com redução média de 16,3% no custo global 1–64. Apenas no grupo 1–8 a Busca Tabu obtém menor custo. Nos demais grupos, incluindo o de maior escala (57–64), o Modelo de Atenção supera a Busca Tabu.

O Modelo de Atenção apresenta o maior tempo médio agregado entre os métodos comparados, sendo cerca de 2,17 vezes mais lento que a Busca Tabu e 3,79 vezes mais lento que o algoritmo genético. Entretanto, o tempo médio por ciclo é de apenas 5,95 s, equivalente a 0,99% da janela disponível de 600 s. Portanto, embora o método demande mais processamento agregado, ele permanece compatível com o replanejamento dinâmico exigido pelo DPDP.

Cabe ressaltar que o foco deste trabalho é avaliar a viabilidade da arquitetura de atenção, não esgotar a comparação com todo o estado da arte do problema.

5. Conclusões

Este trabalho apresentou uma abordagem híbrida para o DPDP baseada em Modelo de Atenção, Aprendizado por Reforço e busca local. A proposta foi motivada pela dificuldade de tomar decisões de despacho em ambientes dinâmicos, nos quais pedidos surgem ao longo do tempo e as rotas precisam respeitar restrições físicas e operacionais. Para lidar com esse cenário, o modelo incorporou atributos temporais, máscaras de viabilidade e uma etapa de refinamento local, buscando combinar generalização aprendida com melhoria heurística das soluções.

Os resultados obtidos no benchmark ICAPS 2021 indicam que a abordagem proposta reduz o custo médio em 98,7% em relação ao Round-Robin e em 16,3% em relação à Busca Tabu na média global 1–64. Em comparação com o algoritmo genético, o Modelo de Atenção apresenta custo médio 31,2% menor nos grupos 1–56, vencendo em seis dos sete grupos considerados nessa média. Entretanto, no grupo 57–64, o algoritmo genético obtém melhor desempenho, fazendo com que a média global 1–64 da abordagem proposta fique 4,17% acima da média do genético. Esse resultado indica que a política neural é competitiva na maioria das escalas avaliadas, mas ainda depende de mecanismos de refinamento mais robustos para instâncias de maior porte.

Como contribuição principal, este trabalho mostra que uma política baseada em atenção pode ser aplicada a um problema dinâmico de coleta e entrega, no qual as decisões são tomadas repetidamente sob informação parcial e com pedidos revelados ao longo do tempo. A formulação proposta combina representação neural das relações entre pedidos e veículos, máscaras de viabilidade operacional e refinamento local das rotas, oferecendo uma alternativa aprendida para orientar o despacho em cenários dinâmicos.

Como trabalhos futuros, pretende-se ampliar o treinamento com instâncias sintéticas mais diversas e investigar mecanismos de integração mais fortes entre o modelo de atenção e a busca local. Em particular, os resultados nas instâncias 57–64 indicam que a política neural deve ser combinada com operadores de vizinhança mais expressivos, estratégias de múltiplas partidas ou

mecanismos adaptativos de seleção de vizinhanças para preservar qualidade em grande escala, além de uma comparação mais aprofundada com outras heurísticas especializadas nesse regime.

Agradecimentos

O presente trabalho foi realizado com o apoio da Fundação de Amparo à Pesquisa do Estado de Minas Gerais (FAPEMIG), por meio do processo nº APQ-06907-25, Edital Compete Minas.

Referências

- Applegate, D. L., Bixby, R. E., Chvátal, V., e Cook, W. J. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, Princeton, NJ.
- Bahdanau, D., Cho, K., e Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Bouros, P., Sacharidis, D., Dalamagas, T., e Sellis, T. (2011). Dynamic pickup and delivery with transfers. In *Advances in Spatial and Temporal Databases*, volume 6849 of *Lecture Notes in Computer Science*, p. 112–129. Springer.
- Cai, J., Zhu, Q., e Lin, Q. (2022). Variable neighborhood search for a new practical dynamic pickup and delivery problem. *Swarm and Evolutionary Computation*, 75:101182.
- Cai, J., Zhu, Q., Lin, Q., Ma, L., Li, J., e Ming, Z. (2023). A survey of dynamic pickup and delivery problems. *Neurocomputing*, 554:126631.
- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., e Zagoruyko, S. (2020). End-to-end object detection with transformers. In *Computer Vision – ECCV 2020*, volume 12346 of *Lecture Notes in Computer Science*, p. 213–229. Springer.
- Chaudhari, S., Mithal, V., Polatkan, G., e Ramanath, R. (2021). An attentive survey of attention models. *ACM Trans. Intell. Syst. Technol.*, 12(5). ISSN 2157-6904.
- Chen, J., Umrawal, A. K., Lan, T., e Aggarwal, V. (2021). Deepfreight: A model-free deep-reinforcement-learning-based algorithm for multi-transfer freight delivery. In *Proceedings of the Thirty-First International Conference on Automated Planning and Scheduling (ICAPS)*, p. 510–518. Association for the Advancement of Artificial Intelligence.
- Cordeau, J.-F., Iori, M., Laporte, G., e Salazar González, J. J. (2010). A branch-and-cut algorithm for the pickup and delivery traveling salesman problem with lifo loading. *Networks*, 55(1):46–59.
- Cordeiro, E. A. e Pitombeira-Neto, A. R. (2023). Deep reinforcement learning for the dynamic vehicle dispatching problem: An event-based approach.
- Dantzig, G. B. e Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1): 80–91.
- Devlin, J., Chang, M.-W., Lee, K., e Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, p. 4171–4186. Association for Computational Linguistics.

- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., e Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*.
- Garey, M. R. e Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, New York.
- Guo, M.-H., Xu, T.-X., Liu, J.-J., Liu, Z.-N., Jiang, P.-T., Mu, T.-J., Zhang, S.-H., Martin, R. R., Cheng, M.-M., e Hu, S.-M. (2022). Attention mechanisms in computer vision: A survey. *Computational Visual Media*, 8:331–368.
- Hao, J., Lu, J., Li, X., Tong, X., Xiang, X., Yuan, M., e Zhuo, H. H. (2022). Introduction to the dynamic pickup and delivery problem benchmark - icaps 2021 competition.
- Jiang, T.-H. e Chang, Y.-C. (2025). Machine learning-enhanced last-mile delivery optimization: Integrating deep reinforcement learning with queueing theory for dynamic vehicle routing. *Applied Sciences*, 15(21):11320.
- Karami, F., Vancroonenburg, W., e Berghe, G. V. (2020). A periodic optimization approach to dynamic pickup and delivery problems with time windows. *Journal of Scheduling*, 23(6):711–731.
- Kool, W., van Hoof, H., e Welling, M. (2019). Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR)*.
- Li, H. e Lim, A. (2001). A metaheuristic for the pickup and delivery problem with time windows. In *Proceedings of the 13th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, p. 160–167. IEEE.
- Li, X., Luo, W., Yuan, M., Wang, J., Lu, J., Wang, J., Lü, J., e Zeng, J. (2021). Learning to optimize industry-scale dynamic pickup and delivery problems. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, p. 2511–2522. IEEE.
- Lin, S. (1965). Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44(10):2245–2269.
- Liu, S., Tan, P. H., Kurniawan, E., Zhang, P., e Sun, S. (2018). Dynamic scheduling for pickup and delivery with time windows. In *2018 IEEE 4th World Forum on Internet of Things (WF-IoT)*, p. 767–770. IEEE.
- Lowerre, B. T. (1976). *The HARPY Speech Recognition System*. PhD thesis, Carnegie Mellon University.
- Ma, H. e Yang, T. (2025). Improved adaptive large neighborhood search combined with simulated annealing (ialns-sa) algorithm for vehicle routing problem with simultaneous delivery and pickup and time windows. *Electronics*, 14(12):2375.
- Mladenović, N. e Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100.
- Psaraftis, H. N. (1988). Dynamic vehicle routing problems. In Golden, B. L. e Assad, A. A., editors, *Vehicle Routing: Methods and Studies*, p. 223–248. North-Holland, Amsterdam.

- Ropke, S. e Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4):455–472.
- Savelsbergh, M. W. P. e Sol, M. (1995). The general pickup and delivery problem. *Transportation Science*, 29(1):17–29.
- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265.
- Stoia, S., Laganà, D., e Ohlmann, J. W. (2025). Dynamic pickup-and-delivery for collaborative platforms with time-dependent travel and crowdshipping. *European Journal of Operational Research*, 322(1):70–84.
- Su, Z., Li, W., Li, J., e Cheng, B. (2022). Heterogeneous fleet vehicle scheduling problems for dynamic pickup and delivery problem with time windows in shared logistics platform: Formulation, instances and algorithms. *International Journal of Systems Science: Operations & Logistics*, 9(2):199–223.
- Sutton, R. S. e Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition.
- Ulmer, M. W., Thomas, B. W., Campbell, A. M., e Woyak, N. (2021). The restaurant meal delivery problem: Dynamic pickup and delivery with deadlines and random ready times. *Transportation Science*, 55(1):75–100.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., e Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, p. 5998–6008.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256.
- Zhou, Y., Kong, L., Yan, L., Liu, Y., e Wang, H. (2024). A memetic algorithm for a real-world dynamic pickup and delivery problem. *Memetic Computing*, 16:203–217.